UNIVERSIDADE FEDERAL DE UBERLÂNDIA

PPGEELT – Pós-Graduação em Engenharia Elétrica

LUCAS ALBINO MARTINS

12622EEL006

TRABALHO 07 – RECONHECIMENTO DE DÍGITOS COM PERCEPTRON MULTICAMADAS NA BASE MNIST

Disciplina: EL056 - Redes Neurais Artificiais

Uberlândia

2026

UNIVERSIDADE FEDERAL DE UBERLÂNDIA

PPGEELT – Pós-Graduação em Engenharia Elétrica

LUCAS ALBINO MARTINS

12622EEL006

TRABALHO 07 – RECONHECIMENTO DE DÍGITOS COM PERCEPTRON MULTICAMADAS NA BASE MNIST

Disciplina: EL056 - Redes Neurais Artificiais

Trabalho apresentado ao Programa de Pós-Graduação em Engenharia Elétrica (PPGEELT) da Universidade Federal de Uberlândia como requisito parcial de avaliação da disciplina.

Disciplina: EL056 - Redes Neurais Artificiais.

Professor: Prof. Dr. Keiji Yamanaka.

Uberlândia

2026

RESUMO

Este trabalho apresenta o treinamento de um Perceptron Multicamadas (MLP) para o reconhecimento dos dígitos manuscritos da base MNIST, com 60.000 imagens de treino e 10.000 de teste em 28×28 pixels, convertidas para a representação bipolar. A rede tem uma camada oculta de 100 neurônios com ativação tangente hiperbólica e foi implementada do zero em NumPy, com treinamento por *backpropagation* com momento em mini-lotes. O critério de parada foi atingido em 26 épocas, e a rede obteve 96,93% de acurácia no conjunto de teste. Com 10% dos pixels invertidos, a acurácia média foi de 91,76%, e com 20%, de 76,44%. Sem o termo de momento, a tolerância não foi atingida em 30 épocas. A rede treinada apenas com dígitos manuscritos reconheceu 90% dos dígitos impressos nas cinco fontes do enunciado. No estudo de sensibilidade, a acurácia de teste subiu de 93,43% com 25 neurônios ocultos para 97,21% com 200.

Palavras-chave: redes neurais artificiais; perceptron multicamadas; *backpropagation*; MNIST; reconhecimento de dígitos.

SUMÁRIO

1 Introdução **4**

2 Fundamentação teórica **4**
2.1 Perceptron multicamadas 4
2.2 Backpropagation em mini-lotes 4
2.3 Termo de momento 5

3 Metodologia **5**
3.1 Base de dados e pré-processamento 5
3.2 Arquitetura e hiperparâmetros 5

4 Resultados **6**
4.1 Treinamento 6
4.2 Acurácia no teste e matriz de confusão 7
4.3 Robustez a ruído 7
4.4 Dígitos impressos 8
4.5 Sensibilidade ao número de neurônios ocultos 8

5 Discussão **9**

6 Conclusão **9**

APÊNDICE A – CÓDIGO-FONTE EM PYTHON **11**

1 INTRODUÇÃO

O reconhecimento de dígitos manuscritos é um problema clássico de classificação de padrões. A base MNIST, apresentada por LeCun et al. (1998), tornou-se a referência para comparar classificadores nessa tarefa: ela reúne 70.000 dígitos escritos por centenas de pessoas, normalizados e centralizados em imagens de 28×28 pixels. O mesmo trabalho mostra que um perceptron multicamadas com uma camada oculta, treinado por *backpropagation* (RUMELHART; HINTON; WILLIAMS, 1986), já alcança taxas de erro de poucos por cento.

O enunciado pede o treinamento de uma MLP para reconhecer dígitos. Nesta versão, a base de treinamento é o MNIST, e os dígitos impressos nas cinco fontes do enunciado entram como teste adicional, com dígitos que a rede não viu no treinamento. Os objetivos são implementar a MLP e o algoritmo de treinamento sem bibliotecas de redes neurais e avaliar a acurácia, a robustez a ruído, o efeito do momento, o efeito do número de neurônios ocultos e a generalização para dígitos impressos.

2 FUNDAMENTAÇÃO TEÓRICA

2.1 Perceptron multicamadas

A MLP é uma rede alimentada adiante em que uma ou mais camadas de neurônios ocultos, com ativação não linear, permitem aprender fronteiras de decisão não lineares (HAYKIN, 2009; BRAGA; CARVALHO; LUDERMIR, 2007). Com n entradas x_i, p neurônios ocultos z_j e m saídas y_k, e usando a notação de Fausett (1994), a fase direta é

$$z_j = f\Big(v_{0j} + \sum_{i=1}^{n} x_i v_{ij}\Big), \qquad y_k = f\Big(w_{0k} + \sum_{j=1}^{p} z_j w_{jk}\Big), \tag{1}$$

em que v_{0j} e w_{0k} são os bias e $f(x) = \tanh(x)$, com $f'(x) = 1 - f(x)^2$.

2.2 Backpropagation em mini-lotes

O treinamento minimiza o erro quadrático $E = \frac{1}{2}\sum_k (t_k - y_k)^2$ por descida de gradiente. A regra delta generalizada (RUMELHART; HINTON; WILLIAMS, 1986) calcula os termos de erro

$$\delta_k = (t_k - y_k)\, f'(y_in_k), \qquad \delta_j = f'(z_in_j) \sum_{k=1}^{m} \delta_k w_{jk}. \tag{2}$$

No algoritmo padrão a padrão, os pesos são corrigidos após cada exemplo. Com dezenas de milhares de exemplos, é usual corrigir os pesos com a média dos gradientes de um pequeno lote de B padrões (HAYKIN, 2009), o que reduz o custo computacional e o ruído das atualizações.

2.3 Termo de momento

Cada correção recebe uma fração μ da correção anterior (FAUSETT, 1994; HAYKIN, 2009):

$$\Delta w_{jk}(t) = \frac{\alpha}{B} \sum_{p \in \text{lote}} \delta_k^{(p)} z_j^{(p)} + \mu \, \Delta w_{jk}(t-1), \tag{3}$$

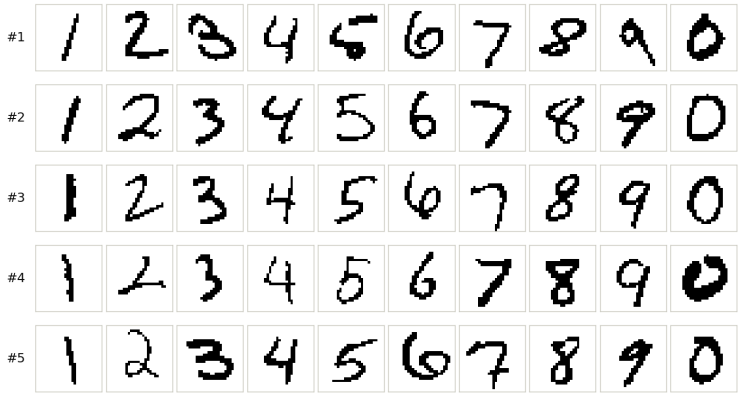
e analogamente para v_{ij}, com $z_0 = x_0 = 1$ para os bias. Com $B = 1$ e $\mu = 0$ obtém-se o algoritmo padrão a padrão sem momento.

3 METODOLOGIA

3.1 Base de dados e pré-processamento

Os quatro arquivos originais do MNIST (formato IDX) foram lidos diretamente com NumPy (HARRIS et al., 2020). As intensidades (0 a 255) foram normalizadas para $[0,1]$ e binarizadas em 0,5: tinta vale $+1$ e fundo, -1. Cada padrão tem 784 entradas, das quais, em média, 103,7 são de tinta. As 5.000 últimas imagens do treino formam o conjunto de validação, usado para escolher a melhor época; restam 55.000 para treino. O conjunto de teste (10.000 imagens) foi usado apenas na avaliação final. A Figura 1 mostra exemplos.

Figura 1 – Cinco exemplos de cada dígito do conjunto de teste, binarizados (preto $= +1$)



Fonte: elaborada pelo autor (2026) a partir da base MNIST.

Para o teste com dígitos impressos, os dez dígitos de cada uma das cinco fontes do enunciado (F1 – DejaVu Sans Bold; F2 – Liberation Serif; F3 – DejaVu Sans ExtraLight; F4 – Kalam, manuscrita; F5 – DejaVu Serif Condensed) passaram pelo pré-processamento do MNIST (LECUN et al., 1998): ajuste a uma caixa de 20×20 pixels com a proporção mantida, centralização pelo centro de massa numa imagem de 28×28 e a mesma binarização (Figura 5).

3.2 Arquitetura e hiperparâmetros

A rede tem 784 entradas, 100 neurônios ocultos e 10 saídas com codificação bipolar *one-hot*; a classe atribuída é a do neurônio de maior saída. O treinamento para quando o erro quadrático médio por padrão, E_{total}/P, fica abaixo da tolerância ou no limite de épocas, e ao final restauram-se os pesos da época de maior acurácia de validação (Tabela 1).

Tabela 1 – Hiperparâmetros da MLP

Parâmetro	Valor
Arquitetura	784–100–10
Taxa de aprendizagem α	0,01
Momento μ	0,5
Tamanho do lote B	32
Tolerância de E_{total}/P	0,05
Máximo de épocas	30
Pesos iniciais	$U(-1/\sqrt{n};\, 1/\sqrt{n})$, semente 2026

Fonte: elaborada pelo autor (2026).

Na inicialização, n é o número de entradas do neurônio, incluindo o bias. Com 784 entradas, uma amplitude fixa como $\pm 0{,}5$ satura a tanh desde o início. Em ensaios preliminares, $\alpha = 0{,}05$ com $\mu = 0{,}9$ fez o treinamento divergir, o que levou aos valores da Tabela 1.

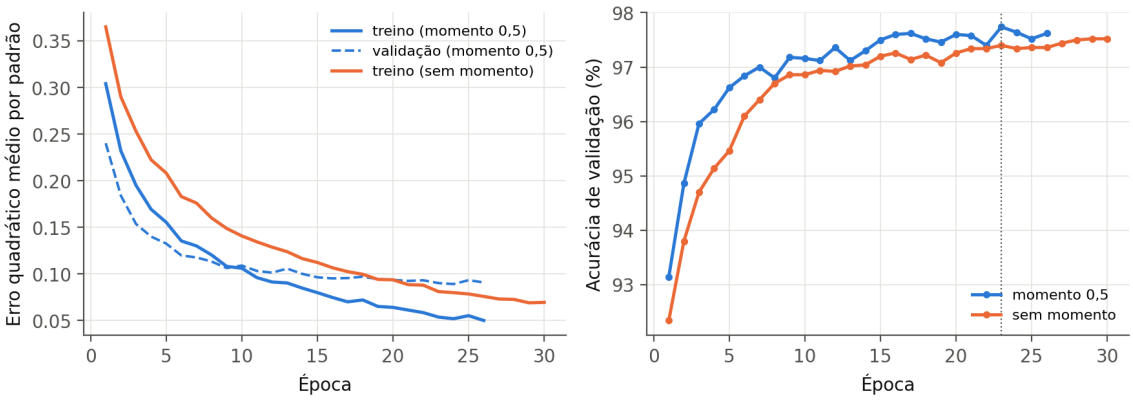
Os experimentos foram: (i) o treinamento principal e o mesmo treinamento sem momento; (ii) acurácia e matriz de confusão no teste; (iii) robustez a ruído, invertendo 5, 10, 15 e 20% dos pixels de cada imagem de teste em 30 repetições por nível; (iv) teste com os 50 dígitos impressos; (v) sensibilidade a 25, 50, 100 e 200 neurônios ocultos, com até 20 épocas cada.

4 RESULTADOS

4.1 Treinamento

O erro quadrático médio por padrão caiu de 0,3041 na primeira época para 0,0500 na época 26, quando a tolerância foi atingida, após 36,8 s de treinamento. A maior acurácia de validação, 97,74%, ocorreu na época 23, cujos pesos foram mantidos. Sem momento, o erro estava em 0,1407 na época 10 e em 0,0936 na época 20, contra 0,1061 e 0,0640 com momento, e terminou as 30 épocas em 0,0694, sem atingir a tolerância (Figura 2). Na validação, o melhor resultado sem momento foi de 97,52%.

Figura 2 – Erro quadrático médio por padrão e acurácia de validação por época

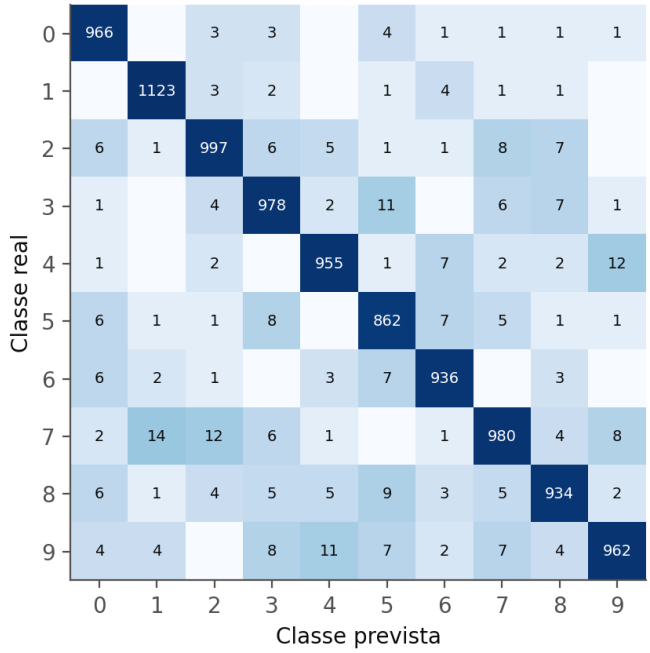


Fonte: elaborada pelo autor (2026).

4.2 Acurácia no teste e matriz de confusão

A acurácia foi de 98,70% no treino e de 96,93% no teste (307 erros em 10.000); a rede sem momento obteve 96,89% no teste. Por classe, a acurácia variou de 95,33% (dígito 7) a 98,94% (dígito 1). As confusões mais frequentes foram 7 → 1 (14 casos), 7 → 2 (12), 4 → 9 (12), 9 → 4 (11) e 3 → 5 (11) (Figura 3).

Figura 3 – Matriz de confusão no conjunto de teste (cores em escala logarítmica)

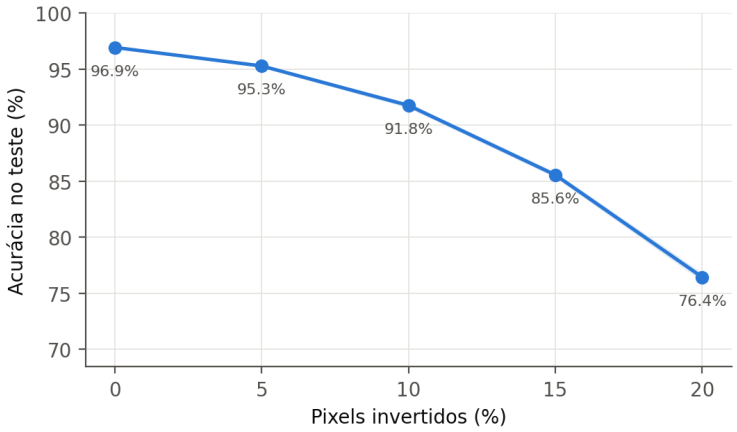


Fonte: elaborada pelo autor (2026).

4.3 Robustez a ruído

A acurácia média caiu para 95,28% com 5% dos pixels invertidos, 91,76% com 10%, 85,56% com 15% e 76,44% com 20%, com desvios padrão inferiores a 0,4 ponto percentual entre as 30 repetições (Figura 4).

Figura 4 – Acurácia média no teste em função do nível de ruído (30 repetições)

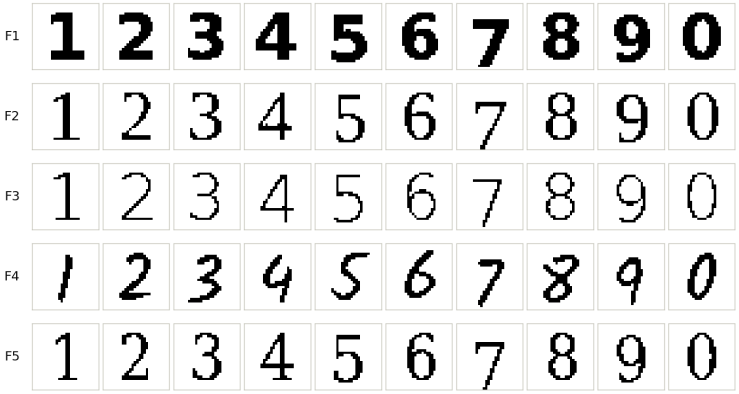


Fonte: elaborada pelo autor (2026).

4.4 Dígitos impressos

A rede reconheceu 45 dos 50 dígitos impressos (90%). As fontes F4 (manuscrita) e F5 (serifada condensada) tiveram 100%, F1 teve 90% (erro 1 → 2) e F2 e F3 tiveram 80% (erros 4 → 5 e 8 → 0 em F2; 4 → 5 e 9 → 8 em F3).

Figura 5 – Dígitos das cinco fontes no formato do MNIST (F1 a F5, de cima para baixo)



Fonte: elaborada pelo autor (2026).

4.5 Sensibilidade ao número de neurônios ocultos

A Tabela 2 mostra que a acurácia cresce com o número de neurônios ocultos, com ganhos cada vez menores, enquanto o número de pesos e o tempo de treinamento crescem aproximadamente na mesma proporção. Com 100 neurônios, o resultado (96,83%) difere do treinamento principal (96,93%) porque o estudo limita o treino a 20 épocas.

Tabela 2 – Efeito do número de neurônios ocultos (até 20 épocas)

Ocultos	Pesos	Teste (%)	Teste com 10% de ruído (%)	Tempo (s)
25	19.885	93,43	85,91	7,9
50	39.760	95,77	88,81	14,5
100	79.510	96,83	91,92	27,8
200	159.010	97,21	92,70	46,8

Fonte: elaborada pelo autor (2026).

5 DISCUSSÃO

A acurácia de 96,93% está no patamar esperado para uma MLP de uma camada oculta no MNIST (LECUN et al., 1998), mesmo com as entradas binarizadas, que descartam os tons de cinza das bordas. Os erros se concentram em pares de traçado semelhante: um 7 sem a barra superior longa se parece com um 1; o 4 fechado se confunde com o 9; o 3 e o 5 diferem apenas na metade superior.

O momento acelerou a redução do erro de treinamento: com $\mu = 0{,}5$, a tolerância foi atingida em 26 épocas, o que não aconteceu em 30 épocas sem momento. A diferença de acurácia no teste, porém, foi pequena (96,93% contra 96,89%). Na prática, o momento encurta o treinamento sem mudar a qualidade final da solução.

Com 784 pixels, a robustez a ruído é menor que a observada com os 50 padrões em grade 12×9 do trabalho anterior: aqui a rede precisa separar milhares de variações de escrita, e as fronteiras entre as classes ficam mais próximas dos exemplos. Inverter 20% dos pixels (157 de 784) cria, em média, cerca de 136 pontos de tinta no fundo, mais do que a tinta de um dígito típico (103,7 pixels), e mesmo assim a rede acerta três de cada quatro imagens.

O teste com dígitos impressos inverte o resultado do trabalho com fontes: lá a fonte manuscrita era a mais difícil, e aqui foi reconhecida integralmente, por se parecer com os dados de treino. Os erros apareceram nas fontes de traço uniforme e com serifas, como o 4 fechado de F2 e F3 confundido com 5. A sensibilidade mostra que 25 neurônios já fornecem uma solução razoável e que dobrar a camada oculta a partir de 100 neurônios traz pouco ganho (0,38 ponto percentual) pelo dobro de pesos.

6 CONCLUSÃO

A MLP implementada do zero e treinada no MNIST alcançou 96,93% de acurácia no teste, com degradação gradual sob ruído (91,76% com 10% dos pixels invertidos). O momento reduziu o tempo até a convergência, e o número de neurônios ocultos mostrou retornos decrescentes a partir de 100. A rede, treinada apenas com dígitos manuscritos, reconheceu 90% dos dígitos impressos nas cinco fontes do enunciado. Como continuidade, seria possível usar as intensidades em tons de cinza, ampliar a base com pequenas deformações dos dígitos ou acrescentar dígitos impressos ao treino.

REFERÊNCIAS

BRAGA, A. de P.; CARVALHO, A. P. d. L. F. d.; LUDERMIR, T. B. *Redes Neurais Artificiais: Teoria e Aplicações*. 2. ed. Rio de Janeiro: LTC, 2007.

FAUSETT, L. *Fundamentals of Neural Networks: Architectures, Algorithms, and Applications*. Englewood Cliffs, NJ: Prentice-Hall, 1994.

HARRIS, C. R. et al. Array programming with NumPy. *Nature*, v. 585, n. 7825, p. 357–362, 2020.

HAYKIN, S. *Neural Networks and Learning Machines*. 3. ed. Upper Saddle River, NJ: Pearson Prentice Hall, 2009.

LECUN, Y. et al. Gradient-based learning applied to document recognition. *Proceedings of the IEEE*, v. 86, n. 11, p. 2278–2324, 1998.

RUMELHART, D. E.; HINTON, G. E.; WILLIAMS, R. J. Learning representations by back-propagating errors. *Nature*, v. 323, n. 6088, p. 533–536, 1986.

APÊNDICE A – CÓDIGO-FONTE EM PYTHON

Arquivo `trabalho07_mnist.py`, executado com `python trabalho07_mnist.py`.

```python
# -*- coding: utf-8 -*-
"""
Trabalho 07 - Reconhecimento de Dígitos com Perceptron Multicamadas (MLP)
Versão com a base MNIST

Disciplina : EL056 - Redes Neurais Artificiais (PPGEELT/UFU)
Professor  : Prof. Dr. Keiji Yamanaka
Aluno      : Lucas Albino Martins (12622EEL006)

O programa:
  1. carrega a base MNIST (60.000 imagens de treino e 10.000 de teste,
     28 x 28 pixels) a partir dos arquivos .gz originais e converte as
     imagens para representação bipolar (-1/+1);
  2. implementa do zero, em NumPy, uma MLP com uma camada oculta, ativação
     tangente hiperbólica e treinamento por backpropagation com momento
     (padrão a padrão ou em mini-lotes);
  3. executa os experimentos: curvas de erro, acurácia e matriz de
     confusão no conjunto de teste, robustez a ruído, comparação com e sem
     momento, sensibilidade ao número de neurônios ocultos e um teste de
     generalização com dígitos impressos em cinco fontes TrueType;
  4. salva figuras e todos os resultados numéricos em figuras/.

Uso:
    python trabalho07_mnist.py              # executa tudo
    python trabalho07_mnist.py --rapido     # versão reduzida (para testes)
"""

import argparse
import gzip
import json
import os
import time
import urllib.request
import warnings

import numpy as np
from PIL import Image, ImageDraw, ImageFont

import matplotlib
matplotlib.use("Agg")                       # gera figuras sem abrir janelas
import matplotlib.pyplot as plt

# ============================================================================
# CONFIGURAÇÃO GERAL (altere aqui)
# ============================================================================

PASTA_SCRIPT = os.path.dirname(os.path.abspath(__file__)) if "__file__" in globals() else os.getcwd()
PASTA_MNIST = os.path.join(PASTA_SCRIPT, "dados", "mnist")

# Arquivos originais do MNIST (formato IDX compactado) e espelhos para
# download automático, caso não estejam na pasta dados/mnist/.
ARQUIVOS_MNIST = {
    "X_treino": "train-images-idx3-ubyte.gz",
    "y_treino": "train-labels-idx1-ubyte.gz",
    "X_teste": "t10k-images-idx3-ubyte.gz",
    "y_teste": "t10k-labels-idx1-ubyte.gz",
}
ESPELHOS_MNIST = [
    "https://ossci-datasets.s3.amazonaws.com/mnist",
    "https://storage.googleapis.com/cvdf-datasets/mnist",
    "https://raw.githubusercontent.com/fgnt/mnist/master",
]

LADO = 28                           # imagens 28 x 28 = 784 entradas
LIMIAR = 0.5                        # intensidade (0-1) mínima para o pixel virar +1
N_VALIDACAO = 5000                  # últimas imagens do treino separadas para validação

# ---- Hiperparâmetros da MLP ----
N_OCULTOS = 100                     # neurônios na camada oculta
```

```
ALFA = 0.01                       # taxa de aprendizagem
MOMENTO = 0.5                     # termo de momento (0 desativa)
TAM_LOTE = 32                     # padrões por atualização (1 = padrão a padrão)
TOLERANCIA = 0.05                 # erro quadrático médio por padrão para parada
MAX_EPOCAS = 30                   # limite de épocas
SEMENTE = 2026                    # semente global (reprodutibilidade)
# Pesos iniciais ~ U(-1/sqrt(n), +1/sqrt(n)), n = nº de entradas do neurônio
# (incluindo o bias). Com 784 entradas, uma amplitude fixa como 0,5
# saturaria a tanh logo no início do treinamento.

# ---- Experimentos ----
NIVEIS_RUIDO = [0.0, 0.05, 0.10, 0.15, 0.20]     # fração de pixels invertidos
REPETICOES_RUIDO = 30                             # repetições por nível
OCULTOS_SENSIBILIDADE = [25, 50, 100, 200]        # estudo de sensibilidade
EPOCAS_SENSIBILIDADE = 20                         # épocas por configuração
EPOCAS_SEM_MOMENTO = MAX_EPOCAS                   # treino de comparação (momento = 0)

# ---- Teste com dígitos impressos (as cinco fontes do enunciado) ----
FONTES = [
    ("Sem serifa negrito (DejaVu Sans Bold)",
     ["DejaVuSans-Bold.ttf", "LiberationSans-Bold.ttf", "arialbd.ttf", "Arial Bold.ttf"]),
    ("Serifada regular (Liberation Serif)",
     ["LiberationSerif-Regular.ttf", "times.ttf", "Times New Roman.ttf", "DejaVuSerif.ttf"]),
    ("Sem serifa fina (DejaVu Sans ExtraLight)",
     ["DejaVuSans-ExtraLight.ttf", "Poppins-Light.ttf", "segoeuil.ttf", "LiberationSans-Regular.ttf"]),
    ("Manuscrita/cursiva (Kalam)",
     ["Kalam-Regular.ttf", "segoepr.ttf", "LiberationSerif-Italic.ttf", "DejaVuSerif-Italic.ttf"]),
    ("Serifada condensada (DejaVu Serif Condensed)",
     ["DejaVuSerifCondensed.ttf", "DejaVuSerif.ttf", "georgia.ttf", "LiberationSerif-Regular.ttf"]),
]
PASTAS_FONTES = [
    os.path.join(PASTA_SCRIPT, "fontes"),
    "/usr/share/fonts", "/usr/local/share/fonts",
    os.path.expanduser("~/.fonts"), os.path.expanduser("~/.local/share/fonts"),
    "/Library/Fonts", "/System/Library/Fonts", os.path.expanduser("~/Library/Fonts"),
    "C:/Windows/Fonts",
]
DIGITOS = ["1", "2", "3", "4", "5", "6", "7", "8", "9", "0"]   # ordem do enunciado
TAMANHO_FONTE = 200               # tamanho (pt) da renderização em alta resolução

# Cores das figuras (tons sóbrios, adequados à impressão)
COR_PRINCIPAL = "#2a78d6"
COR_SECUNDARIA = "#eb6834"
COR_TEXTO = "#52514e"


# ======================================================================
# 1. BASE DE DADOS MNIST
# ======================================================================

def _baixar(nome, pasta):
    """Tenta baixar um arquivo do MNIST dos espelhos configurados."""
    os.makedirs(pasta, exist_ok=True)
    destino = os.path.join(pasta, nome)
    for base in ESPELHOS_MNIST:
        try:
            print(f"    baixando {nome} de {base} ...")
            urllib.request.urlretrieve(f"{base}/{nome}", destino)
            return destino
        except Exception as erro:                       # tenta o próximo espelho
            warnings.warn(f"Falha ao baixar de {base}: {erro}")
    raise FileNotFoundError(f"Não foi possível obter {nome}. Copie os arquivos do MNIST "
                            f"para {pasta}.")


def _ler_idx(caminho, deslocamento):
    """Lê um arquivo IDX compactado (cabeçalho de 'deslocamento' bytes)."""
    with gzip.open(caminho, "rb") as fp:
        return np.frombuffer(fp.read(), dtype=np.uint8, offset=deslocamento)


def carregar_mnist(pasta=PASTA_MNIST):
    """Devolve (X_treino, y_treino, X_teste, y_teste) com intensidades em [0, 1]."""
```

```
    dados = {}
    for chave, nome in ARQUIVOS_MNIST.items():
        caminho = os.path.join(pasta, nome)
        if not os.path.isfile(caminho):
            caminho = _baixar(nome, pasta)
        if chave.startswith("X"):
            dados[chave] = _ler_idx(caminho, 16).reshape(-1, LADO * LADO).astype(float) / 255.0
        else:
            dados[chave] = _ler_idx(caminho, 8).astype(int)
    return dados["X_treino"], dados["y_treino"], dados["X_teste"], dados["y_teste"]


def bipolarizar(X, limiar=LIMIAR):
    """Binariza as intensidades e converte para a representação bipolar (-1/+1)."""
    return np.where(X >= limiar, 1.0, -1.0)


def adicionar_ruido(X, fracao, rng):
    """Inverte o sinal de uma fração exata dos pixels de cada padrão (ruído)."""
    n_pad, n = X.shape
    k = int(round(fracao * n))
    if k == 0:
        return X.copy()
    # sorteio vetorizado de k posições distintas por padrão
    idx = np.argsort(rng.random((n_pad, n)), axis=1)[:, :k]
    Xr = X.copy()
    np.put_along_axis(Xr, idx, -np.take_along_axis(Xr, idx, axis=1), axis=1)
    return Xr


def codificar_alvos(y, n_classes=10):
    """Codificação bipolar one-hot: +1 na classe correta, -1 nas demais."""
    T = -np.ones((len(y), n_classes))
    T[np.arange(len(y)), y] = 1.0
    return T


# ======================================================================
# 2. DÍGITOS IMPRESSOS NO FORMATO DO MNIST
# ======================================================================

def _indexar_pastas(pastas):
    """Percorre as pastas de fontes uma única vez e devolve {nome_minúsculo: caminho}."""
    indice = {}
    for pasta in pastas:
        if not os.path.isdir(pasta):
            continue
        for raiz, _, arquivos in os.walk(pasta):
            for arq in arquivos:
                if arq.lower().endswith((".ttf", ".otf")):
                    indice.setdefault(arq.lower(), os.path.join(raiz, arq))
    return indice


def localizar_fontes(fontes=FONTES, pastas=PASTAS_FONTES, tamanho=TAMANHO_FONTE):
    """Carrega as fontes configuradas, com aviso e alternativa quando faltarem."""
    indice = _indexar_pastas(pastas)
    carregadas = []
    for rotulo, candidatos in fontes:
        caminho = None
        for i, cand in enumerate(candidatos):
            achado = cand if os.path.isfile(cand) else indice.get(os.path.basename(cand).lower())
            if achado:
                caminho = achado
                if i > 0:
                    warnings.warn(f"Fonte preferida '{candidatos[0]}' não encontrada para "
                                  f"'{rotulo}'; usando alternativa '{cand}'.")
                break
        if caminho is None:
            warnings.warn(f"Nenhuma fonte encontrada para '{rotulo}'; usando a fonte padrão do Pillow.")
            fonte, caminho = ImageFont.load_default(size=tamanho), "<padrão do Pillow>"
        else:
            fonte = ImageFont.truetype(caminho, tamanho)
```

```
        carregadas.append((rotulo, caminho, fonte))
    return carregadas


def renderizar_mnist(fonte, digito):
    """Renderiza um dígito com o mesmo pré-processamento do MNIST.

    O MNIST normaliza cada dígito para caber numa caixa de 20 x 20 pixels
    (mantendo a proporção) e o centraliza pelo centro de massa numa imagem
    de 28 x 28 (LeCun et al., 1998).
    """
    lado = 3 * TAMANHO_FONTE
    img = Image.new("L", (lado, lado), 0)                    # fundo 0, tinta 255
    ImageDraw.Draw(img).text((lado // 3, lado // 6), digito, font=fonte, fill=255)
    recorte = img.crop(img.getbbox())                        # caixa delimitadora
    w, h = recorte.size
    escala = 20.0 / max(w, h)
    novo = (max(1, round(w * escala)), max(1, round(h * escala)))
    pequeno = np.asarray(recorte.resize(novo, Image.LANCZOS), dtype=float) / 255.0
    pequeno = np.clip(pequeno, 0, 1)

    # centralização pelo centro de massa
    ys, xs = np.indices(pequeno.shape)
    massa = pequeno.sum()
    cy, cx = (ys * pequeno).sum() / massa, (xs * pequeno).sum() / massa
    saida = np.zeros((LADO, LADO))
    y0, x0 = int(round(LADO / 2 - cy)), int(round(LADO / 2 - cx))
    y0 = min(max(y0, 0), LADO - pequeno.shape[0])
    x0 = min(max(x0, 0), LADO - pequeno.shape[1])
    saida[y0:y0 + pequeno.shape[0], x0:x0 + pequeno.shape[1]] = pequeno
    return saida.ravel()


def construir_base_fontes(fontes_carregadas):
    """50 padrões (10 dígitos x 5 fontes) no formato do MNIST, intensidades em [0, 1]."""
    X, y, id_fonte = [], [], []
    for f, (_, _, fonte) in enumerate(fontes_carregadas):
        for d in DIGITOS:
            X.append(renderizar_mnist(fonte, d))
            y.append(int(d))
            id_fonte.append(f)
    return np.array(X), np.array(y), np.array(id_fonte)


# ======================================================================
# 3. REDE MLP (implementação própria em NumPy)
# ======================================================================

class MLP:
    """Perceptron multicamadas com uma camada oculta e ativação tanh.

    V : pesos entrada -> oculta, forma (n_entradas + 1, n_ocultos); linha 0 = bias
    W : pesos oculta -> saída,   forma (n_ocultos + 1, n_saidas);  linha 0 = bias
    """

    def __init__(self, n_entradas, n_ocultos, n_saidas, semente=SEMENTE):
        rng = np.random.default_rng(semente)
        a_v = 1.0 / np.sqrt(n_entradas + 1)
        a_w = 1.0 / np.sqrt(n_ocultos + 1)
        self.V = rng.uniform(-a_v, a_v, (n_entradas + 1, n_ocultos))
        self.W = rng.uniform(-a_w, a_w, (n_ocultos + 1, n_saidas))
        self.rng = rng

    @staticmethod
    def f(x):
        """Função de ativação: tangente hiperbólica (sigmoide bipolar)."""
        return np.tanh(x)

    @staticmethod
    def df(fx):
        """Derivada da tanh escrita em função da própria saída: 1 - f(x)^2."""
        return 1.0 - fx ** 2

```

```
    def propagar(self, X):
        """Fase direta para um lote de padrões (uma linha por padrão)."""
        Z = self.f(X @ self.V[1:] + self.V[0])      # saídas da camada oculta
        Y = self.f(Z @ self.W[1:] + self.W[0])      # saídas da rede
        return Z, Y

    def erro_medio(self, X, T, lote=10000):
        """Erro quadrático médio por padrão: E_total / P, E_total = 1/2 soma (t - y)^2."""
        E = 0.0
        for i in range(0, len(X), lote):
            _, Y = self.propagar(X[i:i + lote])
            E += 0.5 * float(np.sum((T[i:i + lote] - Y) ** 2))
        return E / len(X)

    def classificar(self, X, lote=10000):
        """Classe = índice do neurônio de saída com maior valor."""
        return np.concatenate([np.argmax(self.propagar(X[i:i + lote])[1], axis=1)
                               for i in range(0, len(X), lote)])

    def copiar_pesos(self):
        return self.V.copy(), self.W.copy()

    def treinar(self, X, T, X_val=None, y_val=None, alfa=ALFA, momento=MOMENTO,
                tam_lote=TAM_LOTE, tolerancia=TOLERANCIA, max_epocas=MAX_EPOCAS,
                verboso=False):
        """Backpropagation com momento, em mini-lotes (tam_lote = 1 -> padrão a padrão).

        Parada: erro quadrático médio por padrão < tolerância ou máximo de épocas.
        Se houver conjunto de validação, os pesos da época de maior acurácia de
        validação são restaurados ao final.
        Retorna um dicionário com o histórico por época.
        """
        dV_ant = np.zeros_like(self.V)
        dW_ant = np.zeros_like(self.W)
        hist = {"erro_treino": [], "erro_val": [], "acc_val": [], "tempo": []}
        melhor_acc, melhores_pesos, melhor_epoca = -1.0, None, 0
        t0 = time.time()
        for epoca in range(1, max_epocas + 1):
            ordem = self.rng.permutation(len(X))
            for ini in range(0, len(X), tam_lote):
                idx = ordem[ini:ini + tam_lote]
                x, t = X[idx], T[idx]
                # --- fase direta ---
                z, yv = self.propagar(x)
                # --- retropropagação do erro ---
                delta_k = (t - yv) * self.df(yv)                 # termos de erro da saída
                delta_j = (delta_k @ self.W[1:].T) * self.df(z)  # termos de erro da oculta
                # --- correções (gradiente médio do lote + momento) ---
                n = len(idx)
                gW = np.vstack([delta_k.sum(axis=0), z.T @ delta_k]) / n
                gV = np.vstack([delta_j.sum(axis=0), x.T @ delta_j]) / n
                dW = alfa * gW + momento * dW_ant
                dV = alfa * gV + momento * dV_ant
                self.W += dW
                self.V += dV
                dW_ant, dV_ant = dW, dV

            E = self.erro_medio(X, T)
            hist["erro_treino"].append(E)
            hist["tempo"].append(time.time() - t0)
            if X_val is not None:
                hist["erro_val"].append(self.erro_medio(X_val, codificar_alvos(y_val)))
                acc = float(np.mean(self.classificar(X_val) == y_val))
                hist["acc_val"].append(acc)
                if acc > melhor_acc:
                    melhor_acc, melhores_pesos, melhor_epoca = acc, self.copiar_pesos(), epoca
            if verboso:
                extra = f"  acc_val = {100 * hist['acc_val'][-1]:.2f}%" if X_val is not None else ""
                print(f"  época {epoca:3d}  E/P = {E:.4f}{extra}  ({hist['tempo'][-1]:.0f} s)")
            if E < tolerancia:
                break
        if melhores_pesos is not None:
            self.V, self.W = melhores_pesos
```

```
        hist["melhor_epoca"] = melhor_epoca
        return hist


def acuracia(rede, X, y):
    return float(np.mean(rede.classificar(X) == y))


# ====================================================================
# 4. EXPERIMENTOS
# ====================================================================

def matriz_confusao(y_real, y_prev, n=10):
    M = np.zeros((n, n), dtype=int)
    np.add.at(M, (y_real, y_prev), 1)
    return M


def teste_ruido(rede, X, y, niveis=NIVEIS_RUIDO, repeticoes=REPETICOES_RUIDO, semente=SEMENTE):
    """Acurácia média (e desvio) no conjunto de teste para cada nível de ruído."""
    rng = np.random.default_rng(semente + 1)
    medias, desvios = [], []
    for nivel in niveis:
        reps = 1 if nivel == 0 else repeticoes
        accs = [acuracia(rede, adicionar_ruido(X, nivel, rng), y) for _ in range(reps)]
        medias.append(float(np.mean(accs)))
        desvios.append(float(np.std(accs)))
    return medias, desvios


def sensibilidade_ocultos(Xtr, Ttr, Xval, yval, Xte, yte, lista=OCULTOS_SENSIBILIDADE,
                          epocas=EPOCAS_SENSIBILIDADE):
    """Varia o nº de neurônios ocultos: acurácia de teste, com 10% de ruído e tempo."""
    saida = []
    for n in lista:
        rede = MLP(Xtr.shape[1], n, 10, semente=SEMENTE)
        hist = rede.treinar(Xtr, Ttr, Xval, yval, max_epocas=epocas)
        m, d = teste_ruido(rede, Xte, yte, niveis=[0.10], repeticoes=10)
        r = {
            "n_ocultos": n,
            "n_pesos": int(rede.V.size + rede.W.size),
            "melhor_epoca": hist["melhor_epoca"],
            "acuracia_val": max(hist["acc_val"]),
            "acuracia_teste": acuracia(rede, Xte, yte),
            "acuracia_teste_ruido10": m[0],
            "tempo_s": round(hist["tempo"][-1], 1),
        }
        print(f"    {n:4d} ocultos: teste = {100 * r['acuracia_teste']:.2f}%, "
              f"10% ruído = {100 * m[0]:.2f}%, {r['tempo_s']} s")
        saida.append(r)
    return saida


# ====================================================================
# 5. GRÁFICOS
# ====================================================================

def _estilo():
    plt.rcParams.update({
        "font.size": 10, "axes.edgecolor": COR_TEXTO, "axes.labelcolor": "#0b0b0b",
        "xtick.color": COR_TEXTO, "ytick.color": COR_TEXTO,
        "axes.spines.top": False, "axes.spines.right": False,
        "axes.grid": True, "grid.color": "#e4e3df", "grid.linewidth": 0.6,
        "savefig.dpi": 200, "savefig.bbox": "tight",
    })


def _grade_imagens(imagens, rotulos_linhas, caminho, titulos=None, cmap="gray_r"):
    """Desenha uma grade de imagens 28 x 28 (uma linha por grupo)."""
    n_l, n_c = len(imagens), len(imagens[0])
    fig, eixos = plt.subplots(n_l, n_c, figsize=(n_c * 0.72, n_l * 0.8))
    eixos = np.atleast_2d(eixos)
    for i in range(n_l):
```

```
        for j in range(n_c):
            ax = eixos[i, j]
            ax.imshow(imagens[i][j].reshape(LADO, LADO), cmap=cmap, vmin=-1, vmax=1,
                      interpolation="nearest")
            ax.set_xticks([]); ax.set_yticks([]); ax.grid(False)
            for s in ax.spines.values():
                s.set_visible(True); s.set_color("#c3c2b7"); s.set_linewidth(0.5)
            if titulos is not None:
                ax.set_title(titulos[i][j], fontsize=7, color=COR_TEXTO, pad=2)
        eixos[i, 0].set_ylabel(rotulos_linhas[i], rotation=0, labelpad=14, va="center", fontsize=9)
    fig.tight_layout(pad=0.3)
    fig.savefig(caminho)
    plt.close(fig)


def fig_base(X, y, caminho, por_digito=5):
    """Grade 5 x 10: os primeiros 'por_digito' exemplos de cada dígito (bipolar)."""
    colunas = [np.flatnonzero(y == int(d))[:por_digito] for d in DIGITOS]
    imagens = [[X[colunas[j][i]] for j in range(len(DIGITOS))] for i in range(por_digito)]
    _grade_imagens(imagens, [f"#{i + 1}" for i in range(por_digito)], caminho)


def fig_fontes(X, n_fontes, caminho):
    """Dígitos impressos convertidos para o formato do MNIST (bipolar)."""
    imagens = [list(X[f * 10:(f + 1) * 10]) for f in range(n_fontes)]
    _grade_imagens(imagens, [f"F{f + 1}" for f in range(n_fontes)], caminho)


def fig_curva_erro(hist, hist_sm, caminho):
    fig, eixos = plt.subplots(1, 2, figsize=(9, 3.3))
    ep = np.arange(1, len(hist["erro_treino"]) + 1)
    ep_sm = np.arange(1, len(hist_sm["erro_treino"]) + 1)
    ax = eixos[0]
    ax.plot(ep, hist["erro_treino"], color=COR_PRINCIPAL, lw=1.8, label="treino (momento 0,5)")
    ax.plot(ep, hist["erro_val"], color=COR_PRINCIPAL, lw=1.4, ls="--", label="validação (momento 0,5)")
    ax.plot(ep_sm, hist_sm["erro_treino"], color=COR_SECUNDARIA, lw=1.8, label="treino (sem momento)")
    ax.set_xlabel("Época"); ax.set_ylabel("Erro quadrático médio por padrão")
    ax.legend(fontsize=8, frameon=False)
    ax = eixos[1]
    ax.plot(ep, 100 * np.array(hist["acc_val"]), color=COR_PRINCIPAL, lw=1.8, marker="o", ms=3,
            label="momento 0,5")
    ax.plot(ep_sm, 100 * np.array(hist_sm["acc_val"]), color=COR_SECUNDARIA, lw=1.8, marker="o",
            ms=3, label="sem momento")
    ax.axvline(hist["melhor_epoca"], color=COR_TEXTO, lw=0.8, ls=":")
    ax.set_xlabel("Época"); ax.set_ylabel("Acurácia de validação (%)")
    ax.legend(fontsize=8, frameon=False, loc="lower right")
    fig.tight_layout()
    fig.savefig(caminho)
    plt.close(fig)


def fig_confusao(M, caminho):
    fig, ax = plt.subplots(figsize=(5.2, 4.4))
    # escala logarítmica para que os erros (fora da diagonal) fiquem visíveis
    ax.imshow(np.log1p(M), cmap="Blues")
    ax.grid(False)
    ax.set_xticks(range(10)); ax.set_yticks(range(10))
    ax.set_xlabel("Classe prevista"); ax.set_ylabel("Classe real")
    lim = np.log1p(M).max() / 2
    for i in range(10):
        for j in range(10):
            if M[i, j]:
                ax.text(j, i, M[i, j], ha="center", va="center", fontsize=7,
                        color="white" if np.log1p(M[i, j]) > lim else "#0b0b0b")
    fig.tight_layout()
    fig.savefig(caminho)
    plt.close(fig)


def fig_ruido(niveis, medias, desvios, caminho):
    fig, ax = plt.subplots(figsize=(6, 3.3))
    pct = [100 * n for n in niveis]
    m, d = np.array(medias) * 100, np.array(desvios) * 100
```

```
    ax.fill_between(pct, m - d, np.minimum(m + d, 100), color=COR_PRINCIPAL, alpha=0.15, lw=0)
    ax.plot(pct, m, color=COR_PRINCIPAL, lw=1.8, marker="o", ms=6)
    for x, v in zip(pct, m):
        ax.text(x, v - 1.5, f"{v:.1f}%", ha="center", va="top", fontsize=8, color=COR_TEXTO)
    ax.set_xticks(pct)
    ax.set_ylim(max(0, m.min() - 8), 100)
    ax.set_xlabel("Pixels invertidos (%)")
    ax.set_ylabel("Acurácia no teste (%)")
    fig.savefig(caminho)
    plt.close(fig)


def fig_erros(X, y, prev, caminho, n=20):
    """Exemplos de imagens de teste classificadas incorretamente."""
    idx = np.flatnonzero(prev != y)[:n]
    imagens = [list(X[idx[:10]]), list(X[idx[10:20]])]
    titulos = [[f"{y[i]} -> {prev[i]}" for i in idx[:10]], [f"{y[i]} -> {prev[i]}" for i in idx[10:20]]]
    _grade_imagens(imagens, ["", ""], caminho, titulos=titulos)


# ======================================================================
# 6. PROGRAMA PRINCIPAL
# ======================================================================

def main(pasta_saida=None, rapido=False):
    global MAX_EPOCAS, EPOCAS_SENSIBILIDADE, EPOCAS_SEM_MOMENTO, REPETICOES_RUIDO
    if rapido:                                  # execução reduzida, apenas para testes
        MAX_EPOCAS, EPOCAS_SENSIBILIDADE, EPOCAS_SEM_MOMENTO, REPETICOES_RUIDO = 2, 1, 2, 3
    else:
        EPOCAS_SEM_MOMENTO = MAX_EPOCAS
    _estilo()
    pasta = pasta_saida or os.path.join(PASTA_SCRIPT, "figuras")
    os.makedirs(pasta, exist_ok=True)
    t0 = time.time()

    # ---- 1. Base MNIST ----
    print("[1] Carregando o MNIST ...")
    Xtr_g, ytr_all, Xte_g, yte = carregar_mnist()
    X_all, X_te = bipolarizar(Xtr_g), bipolarizar(Xte_g)
    X_tr, y_tr = X_all[:-N_VALIDACAO], ytr_all[:-N_VALIDACAO]
    X_val, y_val = X_all[-N_VALIDACAO:], ytr_all[-N_VALIDACAO:]
    T_tr = codificar_alvos(y_tr)
    print(f"    treino: {len(X_tr)}  validação: {len(X_val)}  teste: {len(X_te)}  "
          f"entradas: {X_tr.shape[1]}")
    fig_base(X_te, yte, os.path.join(pasta, "fig_base_dados.png"))
    pixels_ativos = float(np.mean(np.sum(X_all > 0, axis=1)))

    # ---- 2. Treinamento principal ----
    print(f"[2] Treinando a MLP 784-{N_OCULTOS}-10 ...")
    rede = MLP(X_tr.shape[1], N_OCULTOS, 10, semente=SEMENTE)
    hist = rede.treinar(X_tr, T_tr, X_val, y_val, max_epocas=MAX_EPOCAS, verboso=True)

    print("    comparação: mesmo treino sem momento ...")
    rede_sm = MLP(X_tr.shape[1], N_OCULTOS, 10, semente=SEMENTE)
    hist_sm = rede_sm.treinar(X_tr, T_tr, X_val, y_val, momento=0.0,
                              max_epocas=EPOCAS_SEM_MOMENTO)
    fig_curva_erro(hist, hist_sm, os.path.join(pasta, "fig_curva_erro.png"))

    # ---- 3. Teste e matriz de confusão ----
    prev = rede.classificar(X_te)
    M = matriz_confusao(yte, prev)
    acc_teste = float(np.mean(prev == yte))
    acc_classe = (np.diag(M) / M.sum(axis=1)).tolist()
    fora = M.copy(); np.fill_diagonal(fora, 0)
    pares = sorted(((int(fora[i, j]), i, j) for i in range(10) for j in range(10) if fora[i, j]),
                   reverse=True)[:5]
    fig_confusao(M, os.path.join(pasta, "fig_confusao.png"))
    fig_erros(X_te, yte, prev, os.path.join(pasta, "fig_erros.png"))
    print(f"    acurácia: treino = {100 * acuracia(rede, X_tr, y_tr):.2f}%  "
          f"teste = {100 * acc_teste:.2f}%  (sem momento: {100 * acuracia(rede_sm, X_te, yte):.2f}%)")

    # ---- 4. Robustez a ruído ----
    print("[3] Teste de robustez a ruído ...")
```

```
    medias, desvios = teste_ruido(rede, X_te, yte, repeticoes=REPETICOES_RUIDO)
    for n, m, d in zip(NIVEIS_RUIDO, medias, desvios):
        print(f"    ruído {100 * n:4.0f}%: acurácia = {100 * m:6.2f}% (dp {100 * d:.2f})")
    fig_ruido(NIVEIS_RUIDO, medias, desvios, os.path.join(pasta, "fig_ruido.png"))

    # ---- 5. Dígitos impressos (cinco fontes) ----
    print("[4] Teste com dígitos impressos em cinco fontes ...")
    fontes = localizar_fontes()
    rotulos = [r for r, _, _ in fontes]
    Xf, yf, idf = construir_base_fontes(fontes)
    Xf = bipolarizar(Xf)
    fig_fontes(Xf, len(fontes), os.path.join(pasta, "fig_fontes_mnist.png"))
    pf = rede.classificar(Xf)
    por_fonte = [float(np.mean(pf[idf == f] == yf[idf == f])) for f in range(len(fontes))]
    erros_fonte = [[f"{r}->{p}" for r, p in zip(yf[idf == f], pf[idf == f]) if r != p]
                   for f in range(len(fontes))]
    acc_fontes = float(np.mean(pf == yf))
    print(f"    geral = {100 * acc_fontes:.0f}%  por fonte = "
          + ", ".join(f"{100 * a:.0f}%" for a in por_fonte))

    # ---- 6. Sensibilidade ao nº de neurônios ocultos ----
    print("[5] Sensibilidade ao número de neurônios ocultos ...")
    sens = sensibilidade_ocultos(X_tr, T_tr, X_val, y_val, X_te, yte, epocas=EPOCAS_SENSIBILIDADE)

    # ---- Salvar resultados ----
    resultados = {
        "configuracao": {
            "base": "MNIST", "n_treino": int(len(X_tr)), "n_validacao": int(len(X_val)),
            "n_teste": int(len(X_te)), "n_entradas": int(X_tr.shape[1]), "limiar": LIMIAR,
            "n_ocultos": N_OCULTOS, "alfa": ALFA, "momento": MOMENTO, "tam_lote": TAM_LOTE,
            "tolerancia": TOLERANCIA, "max_epocas": MAX_EPOCAS, "semente": SEMENTE,
            "niveis_ruido": NIVEIS_RUIDO, "repeticoes_ruido": REPETICOES_RUIDO,
            "epocas_sensibilidade": EPOCAS_SENSIBILIDADE,
            "fontes": [{"rotulo": r, "arquivo": os.path.basename(c)} for r, c, _ in fontes],
        },
        "base": {"pixels_ativos_medio": pixels_ativos,
                 "amostras_por_classe_treino": np.bincount(y_tr).tolist()},
        "treino": {
            "epocas": len(hist["erro_treino"]), "melhor_epoca": hist["melhor_epoca"],
            "erro_treino": hist["erro_treino"], "erro_val": hist["erro_val"],
            "acc_val": hist["acc_val"], "tempo_s": round(hist["tempo"][-1], 1),
            "convergiu_tolerancia": bool(hist["erro_treino"][-1] < TOLERANCIA),
            "acuracia_treino": acuracia(rede, X_tr, y_tr),
            "acuracia_teste": acc_teste,
            "acuracia_por_classe": acc_classe,
            "matriz_confusao": M.tolist(),
            "maiores_confusoes": [{"real": i, "previsto": j, "casos": c} for c, i, j in pares],
            "n_erros_teste": int(np.sum(prev != yte)),
        },
        "sem_momento": {
            "erro_treino": hist_sm["erro_treino"], "acc_val": hist_sm["acc_val"],
            "melhor_epoca": hist_sm["melhor_epoca"],
            "acuracia_teste": acuracia(rede_sm, X_te, yte),
        },
        "ruido": {"niveis": NIVEIS_RUIDO, "acuracia_media": medias, "acuracia_desvio": desvios},
        "fontes_impressas": {"rotulos": rotulos, "acuracia_geral": acc_fontes,
                             "acuracia_por_fonte": por_fonte, "erros_por_fonte": erros_fonte,
                             "matriz_confusao": matriz_confusao(yf, pf).tolist()},
        "sensibilidade_ocultos": sens,
        "tempo_execucao_s": round(time.time() - t0, 1),
    }
    with open(os.path.join(pasta, "resultados.json"), "w", encoding="utf-8") as fp:
        json.dump(resultados, fp, ensure_ascii=False, indent=2)
    print(f"Concluído em {resultados['tempo_execucao_s']} s. Saídas em: {pasta}")
    return resultados


if __name__ == "__main__":
    parser = argparse.ArgumentParser(description="Trabalho 07 - Reconhecimento de dígitos (MNIST) com MLP")
    parser.add_argument("--saida", default=None, help="pasta de saída (padrão: ./figuras)")
    parser.add_argument("--rapido", action="store_true", help="execução reduzida para testes")
    args = parser.parse_args()
    main(args.saida, args.rapido)
```